

Java OOP Assignment (Chapter 9)

9.3 Using Composition Rather Than Inheritance

This approach uses a HAS-A relationship instead of IS-A.

```
public class BasePlusCommissionEmployee {
    private CommissionEmployee employee;
    private double baseSalary;

    public BasePlusCommissionEmployee(CommissionEmployee employee, double baseSalary) {
        this.employee = employee;
        this.baseSalary = baseSalary;
    }

    public double earnings() {
        return employee.earnings() + baseSalary;
    }
}
```

9.4 Software Reuse

Inheritance promotes reuse by allowing classes to share code, reduces development time, ensures consistency, reduces errors, and improves extensibility.

9.5 Student Hierarchy

```
Student
  ■■■ UndergraduateStudent
    ■ ■■■ Freshman
    ■ ■■■ Sophomore
    ■ ■■■ Junior
    ■ ■■■ Senior
  ■■■ GraduateStudent
    ■■■ MastersStudent
    ■■■ DoctoralStudent
```

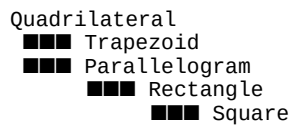
9.6 Shape Hierarchy

```
Shape
  ■■■ TwoDimensionalShape
    ■ ■■■ Circle
    ■ ■■■ Triangle
    ■ ■■■ Quadrilateral
      ■ ■■■ Rectangle
      ■ ■■■ Square
      ■ ■■■ Trapezoid
  ■■■ ThreeDimensionalShape
    ■■■ Sphere
    ■■■ Cube
    ■■■ Cylinder
    ■■■ Cone
```

9.7 protected vs private

Protected allows subclass access but weakens encapsulation. Private enforces strong encapsulation. Best practice is private with getters/setters.

9.8 Quadrilateral Hierarchy



9.9 Code Explanation

super.earnings() calls parent method. @Override indicates method override. super(...) calls parent constructor.

9.10 Code Lines

```
class PieceWorker extends Employee
super.toString();
super(firstName, lastName, ssn);
```

9.11 super in Constructor

Used to initialize superclass variables before subclass execution.

9.12 super in Method

Used to call parent class methods when overridden.

9.13 Using get Methods

Improves encapsulation, consistency, and flexibility.

9.14 Employee Hierarchy Code

```
public class Employee {
    private String firstName, lastName, ssn;

    public Employee(String firstName, String lastName, String ssn) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.ssn = ssn;
    }
}
```

9.15 HourlyEmployee

```
public class HourlyEmployee extends Employee {
    private double hours;
    private double wage;

    public double earnings() {
        if (hours <= 40)
            return hours * wage;
        else
            return (40 * wage) + (hours - 40) * wage * 1.5;
    }
}
```